

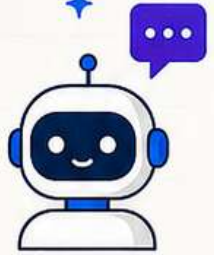


TheLLMEngineer



Agentic AI

with



MCP

(Complete Notes)



MCP Syllabus

&



Top 30

Interview Questions (Easy to Advance)



Learn
Faster



Build
Smarter



Deploy
Confidently



Scale
Effortlessly



MCP

(Model Context Protocol)



CORE SYLLABUS

Module 1: MCP Fundamentals



- What is MCP?
- Why MCP was created
- MCP Architecture
- Client, Server, Host
- MCP Lifecycle

Module 2: MCP Communication



- JSON-RPC 2.0
- Requests & Responses
- Notifications
- Message Flow
- Transport Layer (STDIO, HTTP, SSE)

Module 3: MCP Resources



- Resources
- Resource Templates
- Reading Resources
- Dynamic Resources

Module 4: MCP Tools



- Tool Definition
- Tool Registration
- Tool Invocation
- Tool Parameters
- Tool Results
- Error Handling

Module 5: Prompts



- Prompt Templates
- Dynamic Prompts
- Prompt Arguments
- Prompt Management

Module 6: Building MCP Servers



- Server Setup
- Server Configuration
- Resource Implementation
- Tool Implementation
- Prompt Implementation

Module 7: MCP Clients



- Connecting to MCP Servers
- Discovering Tools
- Calling Resources
- Executing Tools
- Handling Responses

Module 8: Authentication & Security



- Authentication
- Authorization
- Permissions
- Secure Communication
- Best Practices

Module 9: Advanced MCP



- Multi-Server Architecture
- Context Management
- Streaming Responses
- Error Recovery
- Performance Optimization

Module 10: Integration



- MCP with AI Agents
- MCP with LangChain
- MCP with LangGraph
- MCP with Claude Desktop
- MCP with OpenAI SDK

Module 11: Deployment



- Local Deployment
- Docker Deployment
- Remote MCP Servers
- Monitoring
- Logging



Standardize
AI integrations



Connect
tools & data
seamlessly



Secure
by design



Scalable
for any use case



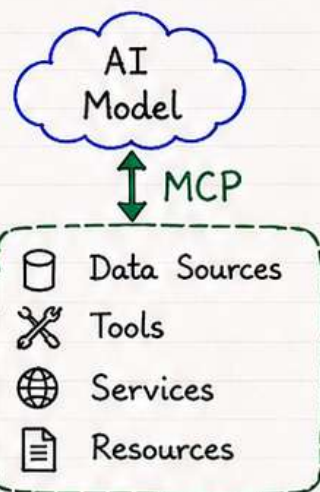
Interoperable
across
ecosystems

✨ MCP (Model Context Protocol) ✨

Module 1: MCP Fundamentals (Part 1)

① What is Model Context Protocol (MCP)?

- MCP is an open protocol that allows AI models (LLMs) to securely connect with external context like data, tools, resources and services.
- It provides a **standardized way** for models to access real-world context beyond their training data.
- MCP acts as a **bridge** between AI models and the outside world.



② Why MCP was created

- Before MCP, every integration with data, tools or systems was **custom-built**, **complex** and hard to maintain.
- Different APIs and formats made it difficult to scale.
- MCP was created to solve these problems by providing:
 - **Standardization**: One protocol for all integrations.
 - **Security**: Controlled & safe access to resources.
 - **Interoperability**: Works across different platforms and tools.
 - **Extensibility**: Easy to add new tools, data sources and capabilities.

Goal of MCP

Make it simple, secure and scalable for AI models to use real-world context.

③ MCP vs APIs vs Function Calling

Feature	MCP	APIs	Function Calling
Definition	Open protocol for AI models to connect with external context (data, tools, services).	Set of endpoints for applications to request & exchange data.	LLM capability to call predefined functions during a conversation.
Purpose	Provide context & capabilities to AI models in a standardized way.	Enable communication between systems or applications.	Allow LLMs to decide when to call a function to get something done.
Who Uses	AI models, agents, developers.	Developers, applications, services.	LLMs (with model support) & developers.
Standardization	Yes, standardized protocol (MCP spec).	No , each API is different.	Not a protocol, model-specific feature.
Scope	Broader - data, tools, services, resources, prompts .	Usually one service or function set.	Limited to functions the model can call.
Flexibility	High - plug & play with many sources.	Medium - depends on API design.	Low to Medium - depends on model & functions.
Example	Connecting an AI model to a DB, File System, Search, Calendar via MCP.	Calling OpenWeather API to get weather data.	Model calling get_weather() function during chat.



Takeaway: MCP makes it easy, secure and standardized for AI models to use real-world context.

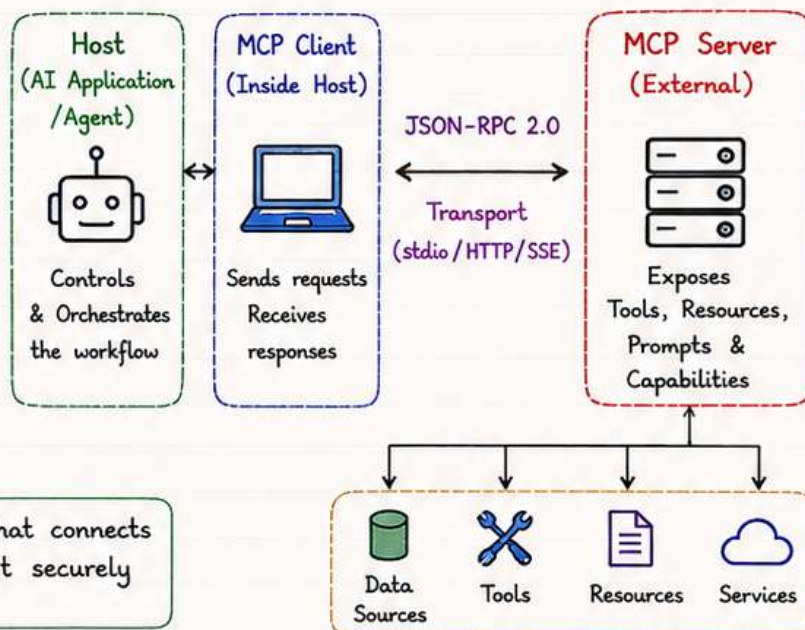


MCP (Model Context Protocol)

Module 1: MCP Fundamentals (Part 2)

④ MCP Architecture

- MCP follows a **client-server** architecture with the **host** (AI application/agent) as the orchestrator.
- It enables secure, **standardized** and efficient access to external context, tools, resources and services.
- Communication happens through **JSON-RPC 2.0** over a transport layer (stdio, HTTP, SSE).



MCP acts as a universal adapter that connects AI models with real-world context securely and consistently.

⑤ Client, Server, Host

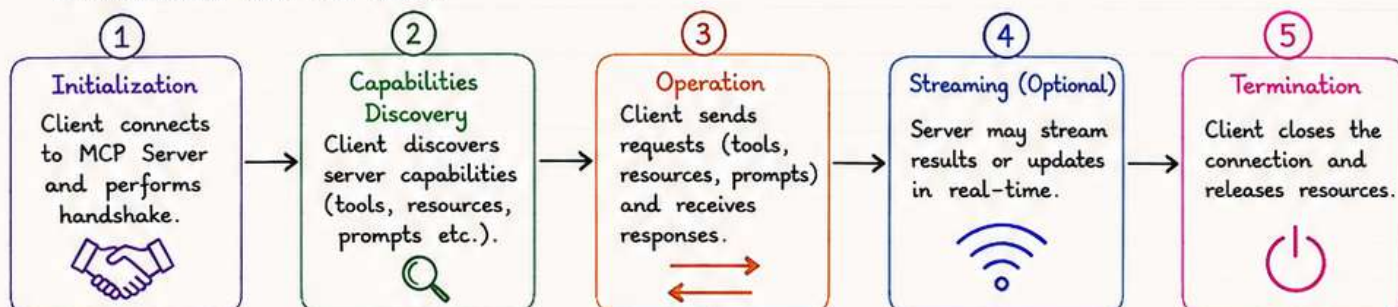
- Host (AI Application / Agent)**
 - The environment where the AI model runs.
 - Orchestrates the workflow and user interactions.
 - Contains the MCP Client.
- Client (MCP Client)**
 - Runs inside the host.
 - Discovers available MCP servers.
 - Sends requests to servers and handles responses.
 - Manages connections and sessions.
- Server (MCP Server)**
 - External service that exposes capabilities.
 - Provides tools, resources, prompts or data.
 - Executes requests and returns results.

Roles at a Glance

Component	Responsibility	Example
Host 	Run AI model, manage workflow, handle user interactions.	Claude Desktop, VS Code with AI extension, Custom Agent
Client 	Connect to MCP servers, send requests, handle responses.	MCP Client Library / SDK
Server 	Expose capabilities (tools, resources, prompts), process requests.	File System Server, GitHub Server, DB Server

⑥ MCP Lifecycle

The lifecycle defines how an MCP connection is established, used and closed.



Key Points

- Lifecycle ensures reliable communication and resource management.
- Supports long-lived sessions and real-time streaming.
- Handles errors, reconnection and graceful shutdown.



✨ MCP (Model Context Protocol) ✨

Module 2: MCP Communication (Part 1)

① JSON-RPC 2.0

- MCP uses **JSON-RPC 2.0** as the underlying **communication protocol**.
- It defines a standard way to send requests, receive responses and send notifications using **JSON** over a transport layer (stdio, HTTP, SSE).
- JSON-RPC 2.0 is **simple, lightweight** and **language-agnostic**.

Core Structure

```
{  
  "jsonrpc": "2.0",    // protocol version  
  "id": 1,              // request id (number or string)  
  "method": "method_name", // method to call  
  "params": { ... }    // parameters (optional)  
}
```



JSON-RPC 2.0 has 3 types of messages:

1) Request 2) Response 3) Notification

② Requests & Responses

- A **Request** is sent by the client to the server to invoke a method.
- A **Response** is returned by the server with the result or error.
- Every request must have a unique **"id"**. The response must include the same **"id"**.

Request Example

```
{  
  "jsonrpc": "2.0",  
  "id": 1,  
  "method": "tools/list",  
  "params": { "cursor": null }  
}
```

Response (Success)

```
{  
  "jsonrpc": "2.0",  
  "id": 1,  
  "result": {  
    "tools": [ { "name": "search" } ]  
  }  
}
```

Response (Error)

```
{  
  "jsonrpc": "2.0",  
  "id": 1,  
  "error": {  
    "code": -32601,  
    "message": "Method not found"  
  }  
}
```



Client

Request (id = 1)

Response (id = 1)



Server



Response must have the same **"id"** as the request.

③ Notifications

- A **Notification** is a message sent from client to server (or server to client) that **does not expect a response**.
- It has no **"id"** field.
- Used for events, updates or one-way messages.

Notification Example

```
{  
  "jsonrpc": "2.0",  
  "method": "notifications/progress",  
  "params": {  
    "progress": 50,  
    "message": "Halfway done"  
  }  
}
```

No Response Expected



Client

Notification

(no id)



Server

Key Takeaways

- ✓ MCP communication is built on **JSON-RPC 2.0**.
- ✓ Requests expect **Responses** (success or error) with the same **id**.
- ✓ **Notifications** are one-way messages and do not expect any response.

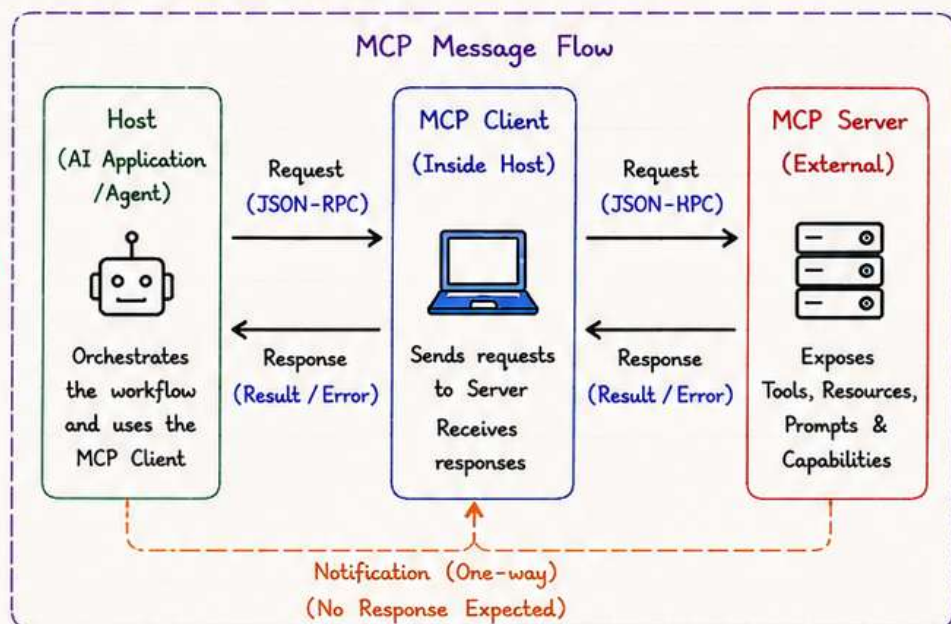


MCP (Model Context Protocol)

Module 2: MCP Communication (Part 2)

④ Message Flow

- The following diagram shows how messages flow between Host, Client and Server in MCP.
- All messages follow **JSON-RPC 2.0** format.
- Communication is request/response based, except **notifications** which are one-way.



Flow Steps

- Host (AI Application/Agent) decides to call a tool or access a resource.
- MCP Client sends a JSON-RPC request to MCP Server.
- MCP Server processes the request and sends back a response.
- For events/updates, Server can send **Notifications** to Client (one-way).
- Client passes the result back to Host for further actions.



Key Points

- All requests expect a response.
- Notifications are one-way messages.
- Each message follows JSON-RPC 2.0.

⑤ Transport Layer (STDIO, HTTP, SSE)

- MCP messages are carried over a transport layer.
- Three main transport options are supported.

Transport	Description	Use Cases	Pros	Cons
STDIO 	Communication happens through the standard input/output streams. (stdin / stdout)	- Local processes - CLI tools - Dev environments	- Simple - Fast - No network overhead	- Only works for local processes - Not suitable for distributed systems
HTTP 	Uses HTTP protocol (usually POST) to send JSON-RPC messages.	- Web services - Remote servers - Cloud deployments	- Widely supported - Works across networks - Easy to debug	- Higher latency - Requires setup (endpoints, auth)
SSE 	Server-Sent Events provides a persistent one-way stream from server to client.	- Real-time updates - Streaming results - Live events	- Real-time push - Lightweight - Works over HTTP	- One-way stream (server → client) - Not for sending requests from client



Summary

MCP Communication uses **JSON-RPC 2.0** messages.

Messages flow between Host, Client and Server.

Transports (STDIO, HTTP, SSE) define how messages are carried.



MCP (Model Context Protocol)

Module 3: MCP Resources

① Resources

- Resources are pieces of data or content exposed by an MCP Server for the Client to access.
- They can be static (files, docs) or dynamic (generated content).
- Resources are identified by a URI.
- Clients can discover, read and use resources as context.

Example Resources		
	file:///readme.md	A markdown file
	db://users/schema	Database schema
	web://example.com/info	Web page content
	api://weather/today	Weather data

② Resource Templates

- Resource Templates define a pattern with parameters to access multiple related resources.
- They use placeholders in the URI.
- Clients provide parameter values to resolve the actual resource.

Template Example

Template URI : file:///docs/{category}/{id}.md

Parameters : {category}, {id}

Example Resolved URIs :

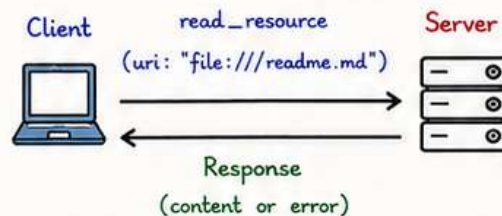
- file:///docs/guide/01.md
- file:///docs/tutorial/intro.md

Benefits

- ✓ Reusable
- ✓ Scalable
- ✓ Flexible
- ✓ Easy discovery

③ Reading Resources

- Clients read resources using the `read_resource` method.
- Server returns the content as text or structured data.
- Large resources may be returned in chunks (streaming).



Response Example

```
{
  "content": "# Welcome to MCP\nThis is a sample document.",
  "mimeType": "text/markdown",
  "encoding": "utf-8"
}
```

④ Dynamic Resources

- Dynamic Resources are generated on demand based on parameters, time or context.
- They are not stored as fixed files.
- Useful for real-time data, calculations or personalized content.

Dynamic Resource Example

URI Template : api://reports/{userId}?type={type}

Parameters : userId, type

Description : Generates a report for a user based on type (summary/detailed).

Client Request

Server Generates

Dynamic Resource Response

Resources at a Glance

Feature	What it is	Example	Purpose
Resources	Data or content exposed by MCP Server	file:///readme.md	Provide context to AI applications
Resource Templates	URI patterns with parameters	file:///docs/{cat}/{id}.md	Access many related resources easily
Reading Resources	Client reads content from Server	read_resource(uri)	Fetch context for LLM / tools
Dynamic Resources	Generated on demand using parameters	api:///reports/{userId}?type=summary	Real-time, personalized or computed content

Key Takeaways

- ✓ Resources give context.
- ✓ Templates make them scalable.
- ✓ Clients read resources to use the content.
- ✓ Dynamic resources enable real-time and flexible data.



Resources are the building blocks of context in MCP, enabling powerful and flexible communication between Clients and Servers.



MCP (Model Context Protocol)

Module 4: MCP Tools (Part 1)

① Tool Definition

- A Tool is a capability that **MCP Server** exposes to Clients.
- Each tool has a **name**, **description**, **input schema** (parameters) and returns **results**.
- Tools help AI models take **actions** or fetch information from external systems.

Key Points

- ✓ Unique name for the tool.
- ✓ Clear description for the model.
- ✓ JSON Schema for input validation.
- ✓ Output format defined by server.

Tool Definition Example

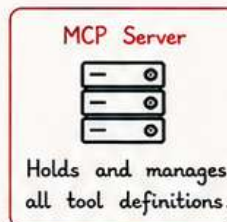
```
{
  "name": "get_weather",
  "description": "Get current weather for a city",
  "inputSchema": {
    "type": "object",
    "properties": {
      "city": { "type": "string", "description": "City name" },
      "unit": { "type": "string", "enum": [ "celsius", "fahrenheit" ],
        "default": "celsius" }
    },
    "required": [ "city" ]
  }
}
```



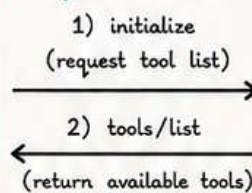
Defines what the tool does and what input it needs.

② Tool Registration

- MCP Server** registers available tools so that **Clients** can discover and use them.
- During initialization, **Server** sends a tools list to the **Client**.
- Clients can list tools anytime.



Registration Flow



Example: tools/list Response

```
{
  "tools": [
    { "name": "get_weather", "description": "Get weather", ... },
    { "name": "search_web", "description": "Search the web", ... }
  ]
}
```

Benefits

- ✓ Discoverability
- ✓ Dynamic updates
- ✓ Consistency

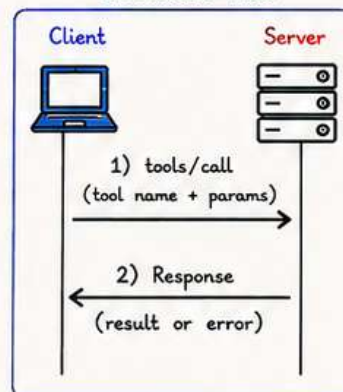
③ Tool Invocation

- Client invokes a tool by sending a request with **tool name** and parameters.
- Server executes the tool and returns the **result** or **error**.
- Responses follow **JSON-RPC 2.0** format.

Tool Invocation Steps

- Client chooses a tool.
- Sends tools/call with arguments.
- Server validates and executes.
- Server returns result or error.

Invocation Flow



Request Example

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": { "city": "Patna", "unit": "celsius" }
  }
}
```

Client calls the tool with arguments.

Success Response Example

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "temperature": 32,
    "unit": "celsius",
    "condition": "Sunny"
  }
}
```

Error Response Example

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32602,
    "message": "Invalid arguments",
    "data": "city is required"
  }
}
```



Key Takeaway : Tools allow MCP Servers to extend capabilities.
Define clearly, register properly, and invoke securely!



✨ MCP (Model Context Protocol) ✨

Module 4 : MCP Tools (Part 2)

④ Tool Parameters

- Tool parameters define what **inputs** the tool expects.
- Defined inside **inputSchema** in the tool definition.
- Follow **JSON Schema** standard.
- Support **required**, **optional**, **default** values, **types**, **enums**, **descriptions**.

Parameter Schema Example

```
"inputSchema": {
  "type": "object",
  "properties": {
    "city": {"type": "string", "description": "City name"},
    "unit": {"type": "string", "enum": [ "celsius", "fahrenheit" ],
      "default": "celsius"},
    "days": {"type": "integer", "description": "Number of days",
      "minimum": 1, "default": 1}
  },
  "required": [ "city" ]
}
```

Parameter Types

- string
- number (integer/float)
- boolean
- array
- object
- enum
- any



Example Tool Call

```
tools/call
{
  "name": "get_weather",
  "arguments": {
    "city": "Patna",
    "unit": "celsius",
    "days": 3
  }
}
```

Best Practices

- ✓ Use clear parameter names.
- ✓ Provide descriptions for each parameter.
- ✓ Use **required** for mandatory inputs.
- ✓ Set reasonable defaults.
- ✓ Validate parameters before execution.

⑤ Tool Results

- After successful execution, the tool returns **results** to the client.
- Results are included in the **Response** message.
- Can return any JSON-serializable data (text, objects, arrays, etc.).
- Keep results clear, structured and meaningful for the client.

Result Format

Success Result Example

```
{
  "content": [
    {
      "type": "text",
      "text": "The current temperature in Patna is 32°C."
    }
  ],
  "isError": false
}
```

Structured Result Example

```
{
  "content": [
    {
      "type": "json",
      "json": {
        "city": "Patna",
        "temperature": 32,
        "unit": "celsius",
        "condition": "Sunny"
      }
    }
  ],
  "isError": false
}
```

Result Content Types

text	
json	
image	
file	
resource	



Key Points

- ✓ Always return a meaningful result.
- ✓ Use structured data when possible.
- ✓ Keep responses concise.
- ✓ Large data can be returned in chunks.

⑥ Error Handling

- If something goes wrong, the server returns an **error** in the Response.
- Follows **JSON-RPC 2.0** error format.
- Helps clients understand what failed and why.
- Errors should be clear and actionable.

Error Response Format (JSON-RPC 2.0)

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "error": {
    "code": -32602,
    "message": "Invalid arguments",
    "data": {
      "details": "city is required"
    }
  }
}
```

Common Error Codes

-32600	Invalid Request
-32601	Method not found
-32602	Invalid params
-32603	Internal error
-32000 to -32099	Server defined errors

Best Practices

- ✓ Return meaningful error messages.
- ✓ Include details in "data" field.
- ✓ Do not expose sensitive information.
- ✓ Log errors on the server.



Tools are the power of MCP. Define good parameters, return useful results and handle errors gracefully for a great client experience.



MCP (Model Context Protocol)

Module 5 : Prompts



Prompts help AI agents interact with users or other systems in a structured and reusable way.

① Prompt Templates

- Prompt Templates are predefined **layouts** with placeholders.
- They ensure consistency and reusability.
- Placeholders are replaced with actual values at runtime.

Template Example

```
{
  "name": "summarize_doc",
  "description": "Summarize a document in given format.",
  "template": "Summarize the following document in {{lang}} language.\n\nDocument:\n{{content}}"
}
```

Rendered Prompt

Summarize the following document in **Hindi** language.

Document:
<actual content here>

Benefits

- ✓ Reusable
- ✓ Consistent
- ✓ Easy to maintain
- ✓ Reduces duplication

② Dynamic Prompts

- Dynamic Prompts are generated on-the-fly based on **context**, **user input** or **external data**.
- They allow personalization and adaptability.
- Useful for real-time and context-aware responses.

Dynamic Prompt Example

```
{
  "name": "greet_user",
  "description": "Greet the user personally.",
  "template": "Hello {{name}}!,\nHow can I help you with {{topic}} today?"
}
```

Example at Runtime

Inputs:

name = "Asgar"
topic = "MCP Learning"

Generated Prompt:

Hello **Asgar!**, **How can** I help you with MCP Learning today?

Use Cases

- Personalized greetings
- Context-aware help
- Dynamic FAQs
- Real-time summaries

③ Prompt Arguments

- Arguments are values passed to fill **placeholders** in templates.
- They can be **required** or **optional**.
- Proper arguments ensure correct and meaningful prompts.

Template with Arguments

```
{
  "name": "explain_concept",
  "description": "Explain a concept with examples.",
  "template": "Explain {{concept}} in simple words with an example for a {{audience}}.",
  "arguments": {
    "concept": { "type": "string", "required": true },
    "audience": { "type": "string", "required": true, "default": "student" }
  }
}
```

Example Call

Arguments Passed:

concept = "MCP"
audience = "beginner"

Final Prompt:

Explain MCP in simple words with an example for a beginner.

Argument Types

- string
- number
- boolean
- array
- object
- enum

④ Prompt Management

- Manage prompts by **creating**, **updating**, organizing and **versioning** them.
- Enables discovery, **access control** and **sharing across teams**.
- Ensures **security**, consistency and better collaboration.

What Can Be Managed?

- ✓ Create / Edit Prompts
- ✓ Organize in Folders/Categories
- ✓ Version Control
- ✓ Access Permissions
- ✓ Tagging & Search
- ✓ Usage Analytics

Example Structure

```
{ Prompts/
  - General/
    - greet_user (v1)
    - faq_template (v2)
  - Documents/
    - summarize_doc (v1)
    - extract_info (v3)
  - Support/
    - troubleshooting (v1)
```

Best Practices

- ✓ Use clear names
- ✓ Add good descriptions
- ✓ Version important prompts
- ✓ Review & update regularly
- ✓ Restrict sensitive prompts



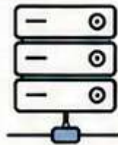
Prompts are the voice of MCP. Templates bring consistency, dynamics bring intelligence, arguments bring flexibility and management brings control.



Great prompts → Better context → Smarter AI agents

MCP (Model Context Protocol)

Module 6: Building MCP Servers (Part 1)



MCP Servers expose capabilities (Resources, Tools, Prompts) to Clients using the MCP protocol.

① Server Setup

- An MCP Server is a process that implements MCP capabilities.
- It listens for MCP requests and provides Resources, Tools and Prompts.
- You can build a server in any language that supports JSON-RPC 2.0.

Basic Setup Steps

- ① Choose a language (Python, Node.js, etc.)
- ② Install MCP SDK / library
- ③ Create a server instance
- ④ Register capabilities (Resources, Tools, Prompts)
- ⑤ Start the server with a transport (STDIO/HTTP/SSE)

Example: Folder Structure

```
my-mcp-server/  
├── server.py  
├── config.py  
├── tools/  
│   └── weather.py  
├── resources/  
│   └── docs.py  
└── prompts/  
    └── summarize.py
```

Minimal Python Server (conceptual)

```
from mcp import Server  
server = Server(name="MyMCPServer", version="1.0.0")  
  
# Register capabilities  
server.register_tool(get_weather)  
server.register_resource(read_docs)  
server.register_prompt(summarize_doc)  
  
# Start server (STDIO transport)  
server.run(transport="stdio")
```

Supported Transports

- STDIO → For local CLI based servers
- HTTP → For web accessible servers
- SSE → For real-time, event driven servers

Key Points

- ✓ Keep server lightweight and focused.
- ✓ Register only the needed capabilities.
- ✓ Choose the right transport for your use case.



② Server Configuration

- Configuration defines how the server behaves and how clients can connect to it.
- Usually provided via config file, environment variables or CLI args.
- Includes server info, transport settings, logging, authentication, timeouts, etc.

Example Configuration (YAML)

```
server:  
  name: "MyMCPServer"  
  version: "1.0.0"  
  description: "Weather and Docs MCP Server"  
  
transport:  
  type: "stdio"      # stdio | http | sse  
  endpoint: null     # used for http/sse  
  
logging:  
  level: "info"      # debug | info | warn | error  
  
auth:  
  enabled: false  
  token: ""  
  
timeouts:  
  request_timeout: 30 # seconds  
  keepalive_interval: 15
```

Configuration Options

Option	Purpose
name	Unique server name
version	Server version
description	What the server does
transport.type	Communication method
transport.endpoint	URL (for HTTP/SSE)
logging.level	Logging verbosity
auth.*	Authentication settings
timeouts.*	Timeout & keepalive

Best Practices

- ✓ Keep config external.
- ✓ Use sensible defaults.
- ✓ Validate config on startup.
- ✓ Never hardcode secrets.



Ways to Load Configuration

- From a YAML / JSON file
- From environment variables
- From command line arguments

Example: Run with Config

```
$ python server.py --config config.yaml  
# or  
$ MCP_CONFIG=config.yaml python server.py
```



Server Setup gets your MCP Server running.
Server Configuration makes it reliable, secure and production-ready.
Good setup + good config = Great MCP Server!



MCP (Model Context Protocol)

Module 6 : Building MCP Servers (Part 2)



MCP Servers expose capabilities (Resources, Tools, Prompts) to Clients.
This page covers implementation of Resources, Tools and Prompts.



③ Resource Implementation

- Resources represent data or content that Clients can read.
- Implement resource handlers to return the content.
- Support both fixed and dynamic resources (using templates).
- Return content as text, JSON, bytes (file) etc.

Example: Resource Implementation (Python)

```
from mcp.server import Server, Resource

server = Server("MyMCPServer")

@server.resource("weather://{city}")
def get_weather(city: str) → str:
    # fetch and return weather info
    data = {"city": city, "temp": 32, "unit": "celsius"}
    return json.dumps(data)

@server.resource("file://readme.md")
def read_readme() → str:
    with open("README.md", "r", encoding="utf-8") as f:
        return f.read()
```

Resource Types

- Static Resource
e.g. file://readme.md
- Template Resource
e.g. weather://{city}
- Dynamic (computed)
e.g. api://status/{id}

Key Points

- ✓ Use clear URI patterns.
- ✓ Validate template params.
- ✓ Return correct content type.
- ✓ Handle errors gracefully.



④ Tool Implementation

- Tools allow Clients to perform actions or execute functions.
- Implement tool handlers that accept parameters and return results.
- Validate input, handle errors and return structured results.

Example: Tool Implementation (Python)

```
from mcp.server import Server, Tool

server = Server("MyMCPServer")

@server.tool("get_weather")
def get_weather(city: str, unit: str = "celsius") → dict:
    # call external API or internal logic
    temp = 32 # dummy
    return {"city": city, "temperature": temp, "unit": unit}

@server.tool("add")
def add(a: int, b: int) → dict:
    return {"result": a + b}
```

Tool Registration Flow

```
Define Tool Function
(with params & logic)
↓
Register with Server
( @server.tool )
↓
Client Invokes Tool
↓
Server Executes &
Returns Result
```

Tool Parameters Example

```
{
  "name": "get_weather",
  "description": "Get current weather for a city",
  "inputSchema": {
    "type": "object",
    "properties": {
      "city": { "type": "string", "description": "City name" },
      "unit": { "type": "string", "enum": [ "celsius", "fahrenheit" ], "default": "celsius" }
    },
    "required": [ "city" ]
  }
}
```

Tool Result Example

```
{
  "content": [
    {
      "type": "json", "json": {
        "city": "Patna",
        "temperature": 32,
        "unit": "celsius"
      }
    }
  ],
  "isError": false
}
```

Best Practices

- ✓ Clear tool names & descriptions.
- ✓ Strong parameter validation.
- ✓ Return structured & meaningful result.
- ✓ Log important actions.



⑤ Prompt Implementation

- Prompts are predefined templates intended to guide the model.
- They improve consistency and reduce repetitive instructions.
- Can be static or dynamic using arguments.

Example: Prompt Implementation (Python)

```
from mcp.server import Server, Prompt

server = Server("MyMCPServer")

@server.prompt("summarize_doc")
def summarize_doc(topic: str, lang: str = "en") → str:
    if lang == "hi":
        return f"निम्नलिखित विषय पर सारांश लिखें: {topic}"
    return f"Summarize the following topic in {lang}: {topic}"
```

Prompt Template Example

```
{
  "name": "summarize_doc",
  "description": "Summarize a document/topic.",
  "arguments": [
    { "name": "topic", "type": "string", "required": true },
    { "name": "lang", "type": "string", "required": false, "default": "en" }
  ]
}
```

Rendered Prompt Example

Summarize the following topic in English:
Topic: Model Context Protocol (MCP)
- (... additional context or instructions ...)

Prompt Use Cases

- Summarization
- Translation
- Explanation
- Q&A Guidance
- Code Generation

Best Practices

- ✓ Keep prompts short & focused.
- ✓ Provide placeholders for inputs.
- ✓ Use defaults sensibly.
- ✓ Version prompts for updates.



Implementing Resources, Tools and Prompts makes your MCP Server powerful and useful.
Good implementations = Better client experience = Smarter AI systems!



MCP (Model Context Protocol)

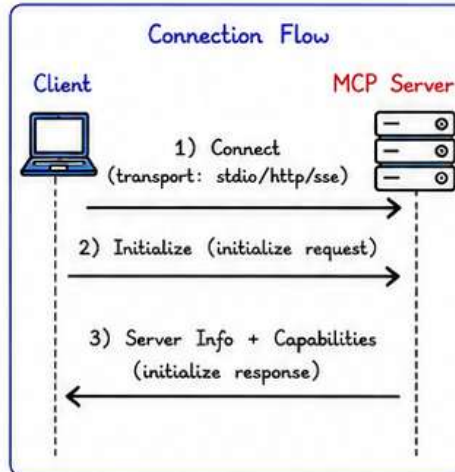
Module 7: MCP Clients (Part 1)



MCP Clients connect to MCP Servers to discover and use Resources, Tools and Prompts.

① Connecting to MCP Servers

- Clients establish a connection to MCP Servers using a supported transport (STDIO, HTTP, SSE).
- After connection, the client performs an initialization handshake.
- The server responds with its capabilities and server information.
- Connection must be secure and reliable.



Example: Initialize Request (JSON-RPC 2.0)

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "client_info": {
      "name": "MyMCPClient",
      "version": "1.0.0"
    }
  }
}
```

Supported Transports

- STDIO → For local CLI based clients
- HTTP → For web / remote servers
- SSE → For real-time streaming

Example: Initialize Response

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "server_info": { "name": "MyMCPServer", "version": "1.0.0" },
    "capabilities": { "resources": true, "tools": true, "prompts": true }
  }
}
```

Key Points

- ✓ Choose correct transport.
- ✓ Handle timeouts & errors.
- ✓ Validate server capabilities.
- ✓ Maintain connection state.

② Discovering Tools

- After connection, clients can discover available Tools exposed by the server.
- Discovery uses the "tools/list" method.
- Server returns a list of tools with names, descriptions, parameters and input schemas.
- Clients use this information to decide which tool to invoke.

Request: tools/list

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/list",
  "params": {}
}
```

Example: tools/list Response

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "tools": [
      {
        "name": "get_weather",
        "description": "Get current weather for a city",
        "inputSchema": {
          "type": "object",
          "properties": {
            "city": { "type": "string", "description": "City name", "unit": { "type": "string", "enum": [ "celsius", "fahrenheit" ], "default": "celsius" } }
        },
        "required": [ "city" ]
      }
    ]
  }
}
```

Tool Info Contains

- ✓ name
- ✓ description
- ✓ inputSchema
- ✓ required fields
- ✓ types and enums

Best Practices

- ✓ Discover tools after connection.
- ✓ Cache tool list when appropriate.
- ✓ Refresh list if server changes.
- ✓ Validate schema before invocation.

Discovery Flow

```
graph TD
    A[Connected to Server] --> B[Send tools/list Request]
    B --> C[Server returns tools]
    C --> D[Client caches tool list]
```

Example Use Case

Agent wants to know what operations it can perform. It asks the server for the tool list and adapts its plan accordingly.



MCP Clients connect, discover and adapt. Good discovery = Smart actions!

Explore first, then act → Smarter AI Applications



MCP (Model Context Protocol) ✨

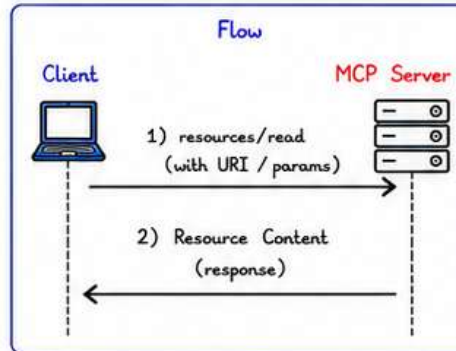
Module 7: MCP Clients (Part 2)



MCP Clients not only discover, but also **call**, **execute** and **handle** everything smoothly using the MCP protocol.

③ Calling Resources

- Clients request **Resources** from the server using `resources/read`.
- Provide the resource **URI** or template + parameters.
- Server returns the **content** of the resource.



Example: Read Resource Request (JSON-RPC 2.0)

```
{
  "jsonrpc": "2.0",
  "id": 10,
  "method": "resources/read",
  "params": {
    "uri": "weather://city",
    "params": {"city": "Patna"}
  }
}
```

Example: Read Resource Response

```
{
  "jsonrpc": "2.0",
  "id": 10,
  "result": {
    "uri": "weather://city?city=Patna",
    "mimeType": "application/json",
    "content": "{\"city\": \"Patna\", \"temp\": 32, \"unit\": \"celsius\"}"
  }
}
```

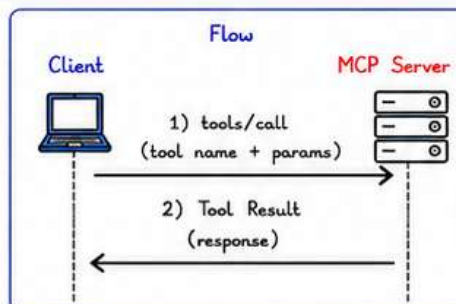
Key Points

- ✓ Use correct URI or fill template parameters.
- ✓ Validate that the resource exists.
- ✓ Handle large content (streaming if needed).
- ✓ Respect **mimeType** and content type.



④ Executing Tools

- Clients invoke **Tools** to perform actions.
- Send tool name and required **parameters**.
- Server executes the tool and returns the **result**.



Example: Call Tool Request (JSON-RPC 2.0)

```
{
  "jsonrpc": "2.0",
  "id": 21,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {"city": "Patna", "unit": "celsius"}
  }
}
```

Example: Tool Result Response

```
{
  "jsonrpc": "2.0",
  "id": 21,
  "result": {
    "content": [
      { "type": "json", "json": {"city": "Patna", "temperature": 32, "unit": "celsius"} }
    ]
  }
}
```

Best Practices

- ✓ Validate input arguments before calling.
- ✓ Provide meaningful tool names and descriptions.
- ✓ Handle long-running tools (async or progress).
- ✓ Return structured results in a consistent format.



⑤ Handling Responses

- Clients must handle all types of **responses** properly.
- Check for **errors**, **timeouts**, and unexpected data.
- Use responses to **update UI**, **context** or take next actions.

Types of Responses

- **Success Response** → result
- **Error Response** → error
- **Notification** → (no id)
- **Progress / Stream** → partial data

Error Response Example

```
{
  "jsonrpc": "2.0",
  "id": 21,
  "error": {
    "code": -32602,
    "message": "Invalid parameters",
    "data": {"details": "city is required"}
  }
}
```

What to Handle

- ✓ Server errors
- ✓ Invalid requests
- ✓ Timeouts / Connection issues
- ✓ Malformed responses
- ✓ Partial / streamed data

Good Client Behavior

- ✓ Retry with backoff (if safe)
- ✓ Show user-friendly messages
- ✓ Log errors for debugging
- ✓ Keep the connection stable

Sample Pseudocode

```
response = send_request(req)
if response.error:
    handle_error(response.error)
else:
    process_result(response.result)
```



Great Clients know how to **Connect**, **Discover**, **Call**, **Execute** and **Handle**!

Robust Clients = **Reliable MCP Applications**



MCP (Model Context Protocol)

Module 8: Authentication & Security



Security in MCP ensures that only trusted Clients can access Servers, and they can access only what they are allowed to do.



① Authentication

- Authentication verifies the identity of Clients (or Users) before allowing access to MCP Servers.
- It answers the question: "Who are you?"
- Common methods: API Keys, OAuth 2.0, Tokens, mTLS.
- After successful authentication, a token or session is issued.

Example: API Key Authentication

Client sends API Key in header:

Authorization: Bearer YOUR_API_KEY

Server verifies the key.

If valid → allow access.

If invalid → reject request.



Key Points



- ✓ Verify identity before any action.
- ✓ Use strong authentication methods.
- ✓ Store secrets securely.
- ✓ Rotate keys/tokens regularly.

② Authorization

- Authorization decides what an authenticated Client is allowed to do.
- It answers the question: "What can you do?"
- Based on roles, scopes or policies.

Example: Role Based Access (RBAC)

User Role: viewer

Allowed Tools: read_weather, search_docs

Denied Tools: delete_user, system_config

Role	Permissions (Tools / Resources)
admin	* (all access)
editor	read, write, update
viewer	read_only
guest	limited_public

Best Practices



- ✓ Use least privilege principle.
- ✓ Do not give more access than required.
- ✓ Review roles and permissions regularly.
- ✓ Log authorization failures.

③ Permissions

- Permissions define access to specific Resources, Tools and Prompts.
- Permissions are enforced by the Server.
- Can be resource-level or action-level.

Example: Permission Structure

Resources

/docs/* → read
/configs/* → read_write

Tools

get_weather → execute
delete_user → denied

Prompts

summarize_doc → use
internal_prompt → denied

Permission Types

- Read → view data
- Write → create / update
- Execute → run tools
- Manage → admin actions
- Deny → explicit block



Key Points



- ✓ Define clear permission policy.
- ✓ Map roles to permissions.
- ✓ Validate permissions for every request.
- ✓ Deny by default.

④ Secure Communication

- MCP Clients and Servers must communicate over secure channels.
- Protects data from interception, tampering and replay attacks.
- Use HTTPS, mTLS, SSE over HTTPS.

Security Measures

- ✓ Use HTTPS for all HTTP traffic.
- ✓ Use TLS 1.2+ for encryption.
- ✓ Prefer mTLS (Mutual TLS) for server-to-server communication.
- ✓ Validate certificates.
- ✓ Use secure headers.
- ✓ Protect against CSRF, XSS, Injection.



Transport Security

- STDIO → Local, secure by nature
- HTTP → Use HTTPS
- SSE → Use SSE over HTTPS
- Webhooks → Verify signatures or tokens

Never Do

- ✗ Send tokens in plain text.
- ✗ Disable SSL verification.
- ✗ Log secrets or tokens.

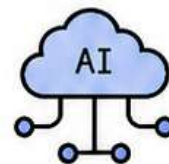


Strong Authentication + Smart Authorization + Correct Permissions
+ Secure Communication = A Secure MCP Ecosystem



MCP (Model Context Protocol)

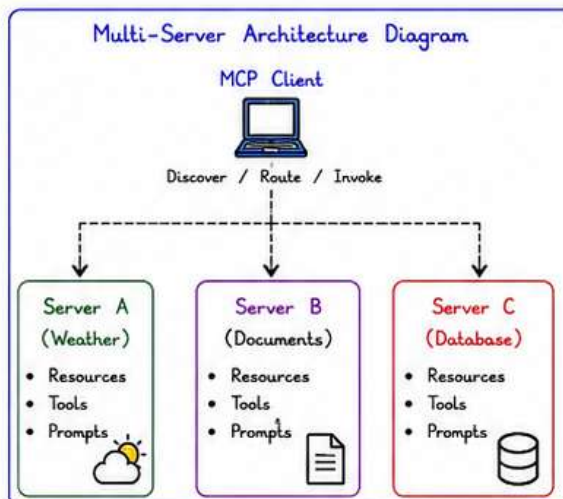
Module 9 : Advanced MCP (Part 1)



Advanced MCP enables **scalable**, intelligent and efficient AI systems through multiple servers and smart **context** handling.

① Multi-Server Architecture

- MCP supports connecting to **multiple** MCP Servers.
- Each server can provide different **Resources**, **Tools** and **Prompts**.
- Clients route requests to the right server based on need.
- Improves **modularity**, **scalability** and **maintainability**.



Key Points



- ✓ Connect to many servers simultaneously.
- ✓ Each server has a specific domain / capability.
- ✓ Client intelligently selects the server.
- ✓ Enables plug-and-play capabilities.
- ✓ Fault isolation between servers.

Example

- Server A → Provides real-time weather data.
- Server B → Manages company documents.
- Server C → Executes database queries.
- Client → Asks a question → Routed to appropriate server.

Benefits



- Scalability → Add new servers easily.
- Modularity → Independent development & deployment.
- Reliability → Failure of one server doesn't affect others.
- Specialization → Servers focus on one domain.

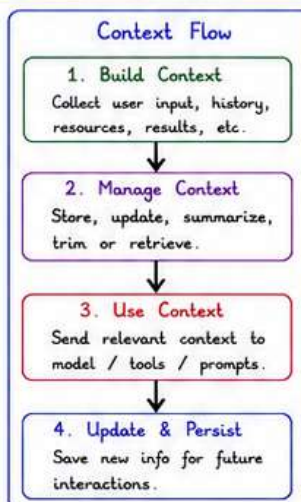
Best Practices



- ✓ Keep servers focused.
- ✓ Use clear naming & descriptions.
- ✓ Monitor server health.
- ✓ Handle server failures gracefully.

② Context Management

- Context is **information** used by the model to understand and respond effectively.
- Good context = better and relevant responses.
- MCP provides ways to create, **store**, **update** and **pass** context across requests.
- Context can be **session-based**, **user-based** or **task-based**.



Types of Context

- Session Context
– Info within a single session.
- User Context
– Info about the user (preferences, history).
- Task Context
– Info related to a specific task or workflow.
- Global Context
– Shared info available across sessions/users.

Techniques

- Summarization → shorten long context.
- Trimming → remove old or less relevant data.
- Retrieval → fetch only needed context.
- Prioritization → keep important info.
- Persistence → store in DB / cache.

Example

- User asks: "What is the weather in Patna tomorrow?"
1. Context includes: previous questions, user location, preferred unit (°C), time.
 2. Add relevant resource from Weather Server.
 3. Send context to model → get accurate answer.
 4. Update context with the new interaction.

Key Points



- ✓ Include only relevant context.
- ✓ Avoid very large context.
- ✓ Keep context up-to-date.
- ✓ Protect sensitive information.
- ✓ Understand context limits of model.

Best Practices

- ✓ Define clear context strategy.
- ✓ Use IDs for sessions & users.
- ✓ Summarize when context grows.
- ✓ Test context impact on quality.
- ✓ Log & monitor context usage.



Multi-Server Architecture gives power to **scale**,
Context Management gives **intelligence** to **respond**!
Together they make MCP systems **Advanced & Powerful**.



MCP (Model Context Protocol)

Module 9: Advanced MCP (Part 2)

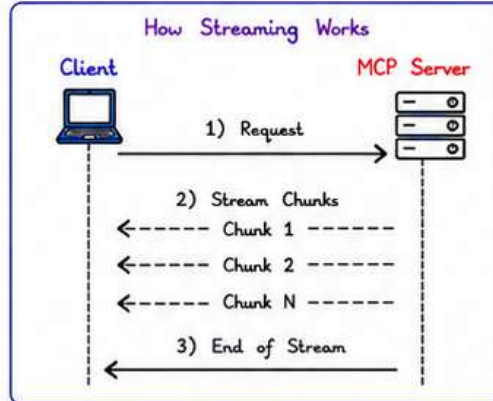


Advanced MCP capabilities help build **fast**, **resilient** and **efficient** systems that deliver great user experience.



③ Streaming Responses

- MCP supports **streaming** of large responses in chunks.
- Useful for long outputs like documents, logs, or AI generations.
- Reduces latency and memory usage.
- Client receives chunks and renders progressively.



Key Points



- ✓ Improves responsiveness.
- ✓ Handles large data efficiently.
- ✓ Lower memory consumption.
- ✓ Real-time user experience.
- ✓ Use when output size is unknown or very large.

Example

AI model generating a long report:

- Request → start streaming
- Server → sends report in chunks
- Client → displays text progressively

Best Practices



- ✓ Use chunked transfer encoding (HTTP).
- ✓ Choose optimal chunk size (not too small, not too large).
- ✓ Handle stream interruptions gracefully.
- ✓ Indicate end of stream clearly.

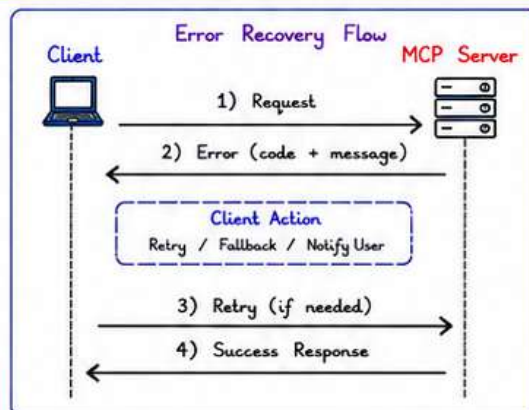
Use Cases



- AI generation
- Large file downloads
- Real-time logs
- Live data feeds

④ Error Recovery

- Errors are normal. Good systems recover and continue.
- MCP defines error codes and messages for consistency.
- Clients should **retry**, **fallback** or **notify** users gracefully.
- Maintain reliability and **user trust**.



Key Points



- ✓ Detect errors early.
- ✓ Use meaningful error codes.
- ✓ Implement retry with backoff.
- ✓ Provide fallback options.
- ✓ Log errors for debugging.

Common Error Types



- 400 Bad Request → Invalid input
- 401 Unauthorized → Auth failed
- 403 Forbidden → No permission
- 404 Not Found → Resource missing
- 429 Too Many Requests → Rate limit
- 500 Internal Error → Server issue

Best Practices

- ✓ Use standard error format.
- ✓ Do not expose sensitive details.
- ✓ Implement exponential backoff.
- ✓ Set max retry limits.
- ✓ Provide user-friendly messages.

⑤ Performance Optimization

- Optimize MCP systems for **speed**, **scalability** and low resource usage.
- Good performance = better user experience + lower cost.
- Focus on network, caching, batching and efficient data handling.



Performance Tips



- ✓ Minimize payload size.
- ✓ Use async / non-blocking calls.
- ✓ Enable compression (gzip).
- ✓ Cache responses where possible.
- ✓ Monitor metrics (latency, throughput).
- ✓ Profile and remove bottlenecks.

Key Metrics to Monitor



- Latency (response time)
- Throughput (requests/sec)
- Error Rate
- Resource Usage (CPU, Memory)

Best Practices

- ✓ Profile before optimizing.
- ✓ Optimize hotspots only.
- ✓ Keep configs environment-specific.
- ✓ Continuously monitor & improve.

Outcome



- Better performance → Happy Users
- Lower Cost → Scalable Systems
- Reliable Experience → Business Growth



Streaming makes it **Fast**, Recovery makes it **Reliable**, Optimization makes it **Scalable** — that's **Advanced MCP!**

